



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Sistemas Distribuidos

Semestre 2027-1 | Grupo 02

Laboratorio S11

Mexi Banco:

Una verdad, varias copias

Persistencia políglota y caché

Alumna: Solís Cisneros Cecilia Ximena

Número de cuenta: 317042333

Profesor: Ing. Oscar Manuel Ruiz Hurtado

Fecha de entrega indicada: 7 de noviembre de 2026

Índice

1. Objetivo **1**

2. Desarrollo **1**

2.1. Preparación del entorno y punto de partida 1

2.2. Incidencias del entorno y recuperación 3

2.3. Caché compartida del saldo en Redis 5

2.4. Invalidación después de una transferencia 6

2.5. Hit ratio y lecturas a PostgreSQL 7

2.6. Latencia con y sin caché 9

2.7. Bitácora de avisos en MongoDB 11

2.8. Consulta de los últimos diez avisos 13

2.9. Índice y planes de ejecución 15

2.10. Evento duplicado e idempotencia 17

2.11. Dato viejo: la caché muestra, pero no autoriza 18

2.12. Reto: funcionamiento con Redis apagado 21

3. Preguntas de cierre **24**

4. Conclusiones **25**

Material de referencia **25**

1. Objetivo

Implementar la persistencia políglota en Mexi Banco, manteniendo el dinero en PostgreSQL, los avisos en MongoDB y una copia temporal del saldo en Redis. A partir de las pruebas, identificar qué beneficio aporta cada almacén y qué ocurre cuando la copia está atrasada o la caché deja de estar disponible.

2. Desarrollo

2.1 Preparación del entorno y punto de partida

Se utilizó la rama `06-poliglota-base` del repositorio de Mexi Banco y se creó la rama de trabajo `trabajo-s11`. Los servicios se ejecutaron con Docker Compose; el servicio de cuenta tuvo tres instancias, para comprobar que la caché era compartida.

La portada conserva la fecha de entrega indicada en la guía. Las evidencias corresponden a las sesiones de trabajo del 6 y 13 de noviembre de 2026, en horario local; varios comandos muestran las fechas del 7 y 14 de noviembre en UTC. Las capturas se conservan con sus marcas de tiempo originales.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ docker compose ps --format "table {{.Name}}\t{{.Status}}"
NAME                          STATUS
mexi-banco-v06-cuenta-1       Up 4 minutes (healthy)
mexi-banco-v06-cuenta-2       Up 4 minutes (healthy)
mexi-banco-v06-cuenta-3       Up 4 minutes (healthy)
mexi-banco-v06-db-1           Up 5 minutes (healthy)
mexi-banco-v06-discovery-1    Up 5 minutes (healthy)
mexi-banco-v06-gateway-1      Up 4 minutes (healthy)
mexi-banco-v06-mongo-1        Up 5 minutes (healthy)
mexi-banco-v06-movimiento-1   Up 4 minutes (healthy)
mexi-banco-v06-notificacion-1 Up 3 minutes (healthy)
mexi-banco-v06-rabbitmq-1     Up 5 minutes (healthy)
mexi-banco-v06-redis-1        Up 5 minutes (healthy)
mexi-banco-v06-spei-1         Up 4 minutes (healthy)
mexi-banco-v06-transferencia-1 Up 3 minutes (healthy)
```

Figura 1: Servicios del laboratorio en el entorno inicial.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli ping
docker compose exec -T mongo mongosh notificacion --quiet --eval 'db.avisos.countDocuments()'
PONG
0
```

Figura 2: Redis respondió PONG y MongoDB tenía cero avisos.

Se crearon las cuentas de Ana, Beto y Carla con saldos de $1000, $500 y $300, respectivamente. Las tres altas respondieron HTTP 201.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ curl -i -X POST http://localhost:8080/api/cuentas \
-H 'Content-Type: application/json' \
-d '{"clabe":"002180000000000001","titular":"Ana","saldoInicial":1000}'

curl -i -X POST http://localhost:8080/api/cuentas \
-H 'Content-Type: application/json' \
-d '{"clabe":"002180000000000002","titular":"Beto","saldoInicial":500}'

curl -i -X POST http://localhost:8080/api/cuentas \
-H 'Content-Type: application/json' \
-d '{"clabe":"002180000000000003","titular":"Carla","saldoInicial":300}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 9230d78b9ca6
Content-Type: application/json
Date: Sat, 07 Nov 2026 04:24:53 GMT

{"id":8,"clabe":"002180000000000001","titular":"Ana","saldo":1000}HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 65ef00a099c7
Content-Type: application/json
Date: Sat, 07 Nov 2026 04:24:55 GMT

{"id":9,"clabe":"002180000000000002","titular":"Beto","saldo":500}HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: b2bc9397e9e1
Content-Type: application/json
Date: Sat, 07 Nov 2026 04:24:57 GMT

{"id":10,"clabe":"002180000000000003","titular":"Carla","saldo":300}
```

Figura 3: Creación de las tres cuentas utilizadas en las pruebas.

2.2 Incidencias del entorno y recuperación

El volumen de PostgreSQL conservaba información de laboratorios anteriores. La columna `evento_id` estaba definida como texto, mientras que el servicio actual esperaba UUID. La primera transferencia devolvió HTTP 500 después de aplicar el cargo de $200 a Ana, pero antes de abonar a Beto.

Antes de corregirlo, se respaldó la base de transferencia y se comprobó que todos los identificadores existentes tenían formato UUID. Después se convirtió la columna con `USING evento_id::uuid`. El registro de la operación confirmó el cargo con la clave `transferencia-22-cargo`. Se habilitó la compensación de esa transferencia mediante una actualización controlada de su estado; el servicio efectuó la devolución y dejó la transferencia 22 en `COMPENSADA`. Ana regresó a $1000 y Beto permaneció en $500.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T db psql -U mexibanco -d transferencia -v ON_ERROR_STOP=1 -c \
"UPDATE transferencia
 SET estado = 'COMPENSACION_PENDIENTE',
     saldo_origen_tras_cargo = 800.00,
     motivo = 'Recuperacion manual tras incompatibilidad de evento_id; cargo confirmado y abono no realizado',
     actualizada = CURRENT_TIMESTAMP
 WHERE id = 22 AND estado = 'INICIADA';"
UPDATE 1

cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/transferencias/22
echo

for clabe in 002180000000000001 002180000000000002; do
  curl -s "http://localhost:8080/api/cuentas/$clabe"
  echo
done
{"id":22,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":200.00,"fecha":"2026-11-07T04:28:54.703899Z","
estado":"COMPENSADA","motivo":"Recuperacion manual tras incompatibilidad de evento_id; cargo confirmado y abono no realizado","saldoOr
igenTrasCargo":800.00,"intentosDeCompensacion":1,"actualizada":"2026-11-07T04:50:11.973399Z","eventoId":"63e87827-b44a-4ab1-b67e-af3fe
18ea7a4"}
{"clabe":"002180000000000001","titular":"Ana","saldo":1000.00,"leidaEn":"2026-11-07T04:50:14.274036310Z"}
{"clabe":"002180000000000002","titular":"Beto","saldo":500.00,"leidaEn":"2026-11-07T04:50:14.527458708Z"}
```

Figura 4: Recuperación del cargo mediante la compensación de la transferencia 22.

Al retomar el trabajo también se presentó un error de permisos al leer el archivo interno de autenticación de RabbitMQ. Se corrigieron el propietario y los permisos, sin incluir esa credencial en la entrega. Los servicios volvieron a estar disponibles.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ docker compose ps -a --format "table {{.Name}}\t{{.Status}}"
NAME                              STATUS
mexi-banco-v06-cuenta-1           Up 21 minutes (healthy)
mexi-banco-v06-cuenta-2           Up 21 minutes (healthy)
mexi-banco-v06-cuenta-3           Up 21 minutes (healthy)
mexi-banco-v06-db-1               Up 22 minutes (healthy)
mexi-banco-v06-discovery-1        Up 22 minutes (healthy)
mexi-banco-v06-gateway-1          Up 21 minutes (healthy)
mexi-banco-v06-mongo-1            Up 22 minutes (healthy)
mexi-banco-v06-movimiento-1       Up 21 minutes (healthy)
mexi-banco-v06-notificacion-1     Up About a minute (healthy)
mexi-banco-v06-rabbitmq-1         Up 3 minutes (healthy)
mexi-banco-v06-redis-1            Up 22 minutes (healthy)
mexi-banco-v06-spei-1             Up 21 minutes (healthy)
mexi-banco-v06-transferencia-1    Up About a minute (healthy)
```

Figura 5: Estado de los servicios después de recuperar RabbitMQ.

Una vez corregido el esquema, la transferencia 23 de Ana a Beto por $200 terminó con estado `COMPLETADA` y HTTP 201. Los saldos quedaron en $800 para Ana y $700 para Beto.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ curl -i -X POST http://localhost:8080/api/transferencias \
-H 'Content-Type: application/json' \
-d '{"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":200}'
echo
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Sat, 07 Nov 2026 04:51:36 GMT

{"id":23,"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":200,"fecha":"2026-11-07T04:51:35.766702770Z","
estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":800.00,"intentosDeCompensacion":0,"actualizada":"2026-11-07T04:51:36.0152232
20Z","eventoId":"7a369222-b67a-4e2c-a774-4af7ab8a816a"}
```

Figura 6: Transferencia inicial completada después de corregir el entorno.

Antes de activar la caché, se realizaron diez consultas del saldo de Ana. Se comparó el número de líneas `leida de Postgres` antes y después del bloque, para no contar consultas de las pruebas anteriores. Hubo diez lecturas y las diez respuestas tuvieron un `leidaEn` distinto. Redis mantuvo cero aciertos y cero fallos, porque todavía no intervenía.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ antes=$(docker compose logs cuenta 2>&1 | grep -c "leida de Postgres")

for i in $(seq 1 10); do
  curl -s http://localhost:8080/api/cuentas/00218000000000001
  echo
done

despues=$(docker compose logs cuenta 2>&1 | grep -c "leida de Postgres")

echo "Consultas realizadas: 10"
echo "Lecturas a Postgres: $((despues-antes))"

docker compose exec -T redis redis-cli INFO stats | grep keyspace
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T04:53:09.514245962Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T04:53:09.780119220Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T04:53:10.018838825Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T04:53:10.262103714Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T04:53:10.513763482Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T04:53:10.745571822Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T04:53:10.973297118Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T04:53:11.162649562Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T04:53:11.437077638Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T04:53:11.673099772Z"}
Consultas realizadas: 10
Lecturas a Postgres: 10
keyspace_hits:0
keyspace_misses:0
```

Figura 7: Diez consultas iniciales produjeron diez lecturas a PostgreSQL.

2.3 Caché compartida del saldo en Redis

Se activó `@EnableCaching` y se configuró la caché `cuentas` con un TTL de 30 segundos, valores JSON de `CuentaVista` y sin guardar valores nulos. A su vez, se utilizó la configuración transaccional para que las escrituras y los borrados de caché esperaran al commit.

La anotación `@Cacheable` se colocó en `ConsultaDeSaldo.consultar`, que atiende la lectura para mostrar el saldo. Se añadió `@CacheEvict` en `cargar` y `abonar`. La lectura que autoriza el cargo, en `CuentaService`, se mantuvo en PostgreSQL.

```
@Cacheable(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
public CuentaVista consultar(String clabe)

@CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
@Transactional
public SaldoAplicado cargar(String clabe, BigDecimal monto, String clave)
```

Con cache-aside, un acierto devuelve la copia sin ejecutar la consulta a PostgreSQL. Cuando no existe la entrada, se consulta la base y se guarda el resultado para las siguientes peticiones.

Se vació únicamente la clave de Ana y se hicieron seis consultas. Las respuestas alternaron entre las tres instancias, pero mantuvieron el mismo `leidaEn`. El conteo registró una lectura a PostgreSQL. El JSON de Redis coincidió con las respuestas y su TTL fue de 23 segundos al consultarlo.

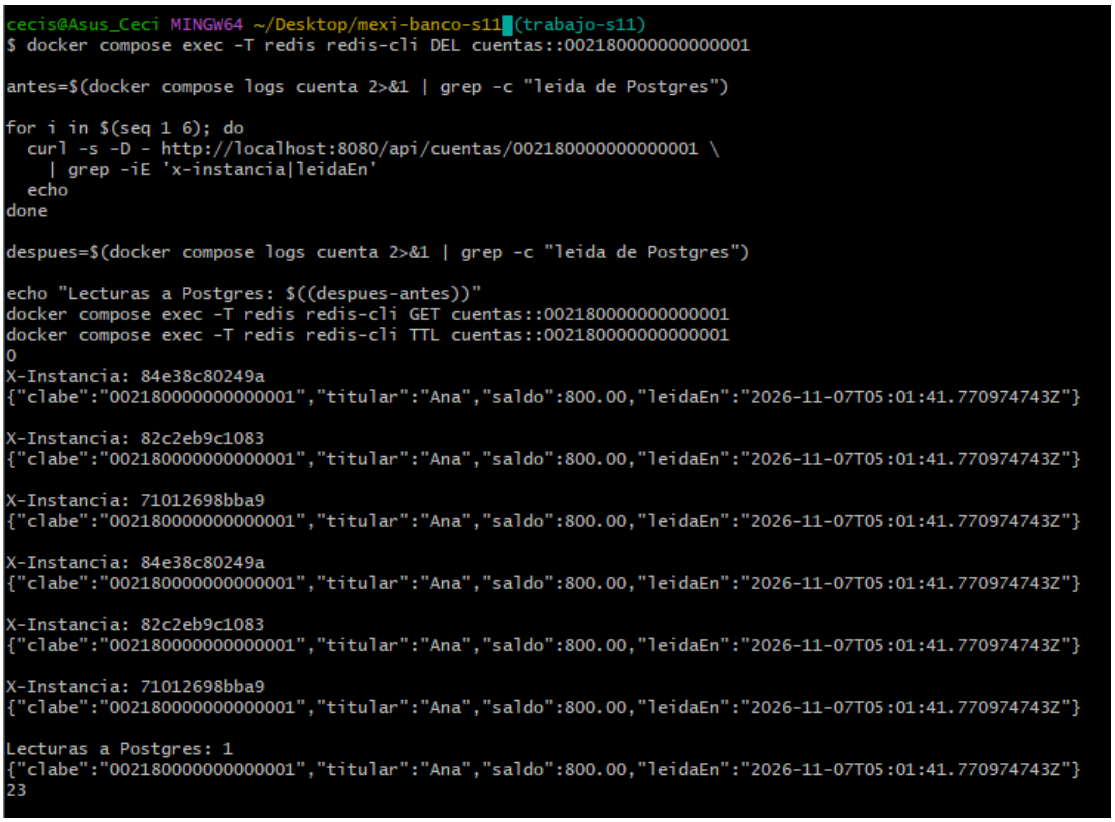


Figura 8: Las tres instancias compartieron la misma copia; seis consultas causaron una lectura.

2.4 Invalidación después de una transferencia

Se consultó el saldo de Ana para llenar la caché. La comprobación `EXISTS` devolvió 1. Después se transfirieron $10 a Beto, con HTTP 201, y la misma comprobación devolvió 0. La siguiente lectura mostró el saldo nuevo de $790 y una nueva hora de lectura.

Con esta prueba se observó que el cambio de saldo no actualizó la copia: la borró. Así, la siguiente consulta reconstruyó el dato desde PostgreSQL.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ # Guardar el saldo actual en cache
curl -s http://localhost:8080/api/cuentas/002180000000000001
echo

# Confirmar que la copia existe: debe dar 1
docker compose exec -T redis redis-cli EXISTS cuentas::002180000000000001

# Transferir 10 pesos de Ana a Beto
curl -s -w '\nHTTP %{http_code}\n' \
-X POST http://localhost:8080/api/transferencias \
-H 'Content-Type: application/json' \
-d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":10}'

# Revisar que la copia se borro: debe dar 0
docker compose exec -T redis redis-cli EXISTS cuentas::002180000000000001

# Consultar el saldo nuevo: esperamos 790
curl -s http://localhost:8080/api/cuentas/002180000000000001
echo
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T05:05:05.938330571Z"}
1
{"id":24,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":10,"fecha":"2026-11-07T05:05:10.131821003Z","e
stado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":790.00,"intentosDeCompensacion":0,"actualizada":"2026-11-07T05:05:10.54973462
5Z","eventoId":"4389649f-411c-47d5-8161-0741cfd9c38a"}
HTTP 201
0
{"clabe":"002180000000000001","titular":"Ana","saldo":790.00,"leidaEn":"2026-11-07T05:05:14.490124510Z"}
```

Figura 9: La entrada desapareció después de la transferencia y se reconstruyó con el saldo actualizado.

2.5 Hit ratio y lecturas a PostgreSQL

Se reiniciaron los contadores de Redis antes de cada escenario. En A se realizaron cien consultas seguidas; en B se hizo una transferencia de $1 de Ana a Beto después de cada diez consultas. Las diez transferencias de B respondieron HTTP 201.

$$\text{Hit ratio} = \frac{\text{hits}}{\text{hits} + \text{misses}} \times 100$$

Escenario	Hits	Misses	Hit ratio	Lecturas PG
A: 100 consultas	99	1	99 %	1
B: consultas y transferencias	100	20	83.33 %	20

En B, los contadores sumaron 120 accesos porque las transferencias también consultan las cuentas de origen y destino por el endpoint de lectura. Esas consultas previas no sustituyen la autorización del cargo, que sigue usando PostgreSQL. Las transferencias invalidaron las copias y provocaron nuevas lecturas. Por eso el porcentaje bajó, aun cuando la caché siguió atendiendo la mayor parte de los accesos.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli DEL cuentas::0021800000000000001
docker compose exec -T redis redis-cli CONFIG RESETSTAT

antes=$(docker compose logs cuenta 2>&1 | grep -c "leida de Postgres")

for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/0021800000000000001
done

despues=$(docker compose logs cuenta 2>&1 | grep -c "leida de Postgres")

echo "Escenario A: 100 consultas sin transferencias"
echo "Lecturas a Postgres: $((despues-antes))"
docker compose exec -T redis redis-cli INFO stats | grep keyspace
0
OK
Escenario A: 100 consultas sin transferencias
Lecturas a Postgres: 1
keyspace_hits:99
keyspace_misses:1
```

Figura 10: Escenario A: 99 aciertos y un fallo.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli DEL cuentas::0021800000000000001
docker compose exec -T redis redis-cli CONFIG RESETSTAT

antes=$(docker compose logs cuenta 2>&1 | grep -c "leida de Postgres")

for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/0021800000000000001

  if (( i % 10 == 0 )); then
    curl -s -o /dev/null -w "Transferencia: HTTP %{http_code}\n" \
      -X POST http://localhost:8080/api/transferencias \
      -H 'Content-Type: application/json' \
      -d '{"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":1}'
  fi
done

despues=$(docker compose logs cuenta 2>&1 | grep -c "leida de Postgres")

echo "Escenario B: 100 consultas y 10 transferencias"
echo "Lecturas a Postgres: $((despues-antes))"
docker compose exec -T redis redis-cli INFO stats | grep keyspace
0
OK
Transferencia: HTTP 201
Transferencia: HTTP 201
Transferencia: HTTP 201
Transferencia: HTTP 201
Transferencia: HTTP 201
Transferencia: HTTP 201
Transferencia: HTTP 201
Transferencia: HTTP 201
Transferencia: HTTP 201
Transferencia: HTTP 201
Escenario B: 100 consultas y 10 transferencias
Lecturas a Postgres: 20
keyspace_hits:100
keyspace_misses:20
```

Figura 11: Escenario B: 100 aciertos y 20 fallos, con diez transferencias completadas.

2.6 Latencia con y sin caché

Se hicieron tres corridas de cien consultas por configuración. Se midió el tiempo total de cada petición con curl. La mediana se calculó con los valores centrales y el p95 con la observación 95 de los cien tiempos ordenados. Cada corrida con caché comenzó borrando la clave de Ana, por lo que incluyó el primer fallo de caché. Para medir sin caché se utilizó `CACHE_TIPO=none`.

Configuración	Corrida	Promedio (ms)	Mediana (ms)	p95 (ms)	Lecturas PG
Con caché	1	20.63	19.43	34.18	1
Con caché	2	18.49	16.90	28.32	1
Con caché	3	18.34	16.68	25.66	1
Sin caché	1	26.56	23.13	42.17	100
Sin caché	2	24.32	22.52	37.33	100
Sin caché	3	23.59	21.73	38.89	100

Las seis corridas tuvieron cero errores HTTP. El promedio de los tres promedios fue de 19.15 ms con caché y 24.82 ms sin ella, una reducción aproximada de 22.84 % en esta prueba local. Este resultado depende de las condiciones de la máquina y no representa una prueba de carga concurrente.

La diferencia más clara estuvo en las lecturas: de cien a una por corrida. Por eso, además de reducir algo la latencia, la caché disminuyó el trabajo de PostgreSQL. Esto sugiere mayor margen para atender otras operaciones, aunque no se midió directamente la capacidad máxima del sistema.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ for corrida in 1 2 3; do
  docker compose exec -T redis redis-cli DEL cuentas::002180000000000001 > /dev/null
  antes=$(docker compose logs cuenta 2>&1 | grep -c "leida de Postgres")

  echo "CON CACHE - corrida $corrida"

  for i in $(seq 1 100); do
    curl -s -o /dev/null -w '%{http_code} %{time_total}\n' \
      http://localhost:8080/api/cuentas/002180000000000001
  done | sort -k2,2n | awk '
    {t[NR]=$2*1000; s+=t[NR]; if ($1 != 200) errores++}
    END {
      printf "Consultas=%d promedio=%.2f ms mediana=%.2f ms p95=%.2f ms errores=%d\n",
        NR, s/NR, (t[50]+t[51])/2, t[95], errores
    }'

  despues=$(docker compose logs cuenta 2>&1 | grep -c "leida de Postgres")
  echo "Lecturas a Postgres: $((despues-antes))"
done
CON CACHE - corrida 1
Consultas=100 promedio=20.63 ms mediana=19.43 ms p95=34.18 ms errores=0
Lecturas a Postgres: 1
CON CACHE - corrida 2
Consultas=100 promedio=18.49 ms mediana=16.90 ms p95=28.32 ms errores=0
Lecturas a Postgres: 1
CON CACHE - corrida 3
Consultas=100 promedio=18.34 ms mediana=16.68 ms p95=25.66 ms errores=0
Lecturas a Postgres: 1
```

Figura 12: Tres corridas con caché: una lectura a PostgreSQL por cada cien consultas.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ for corrida in 1 2 3; do
  antes=$(docker compose logs cuenta 2>&1 | grep -c "leida de Postgres")

  echo "SIN CACHE - corrida $corrida"

  for i in $(seq 1 100); do
    curl -s -o /dev/null -w '%{http_code} %{time_total}\n' \
      http://localhost:8080/api/cuentas/002180000000000001
  done | sort -k2,2n | awk '
    {t[NR]=$2*1000; s+=t[NR]; if ($1 != 200) errores++}
    END {
      printf "Consultas=%d promedio=%.2f ms mediana=%.2f ms p95=%.2f ms errores=%d\n",
        NR, s/NR, (t[50]+t[51])/2, t[95], errores
    }'

  despues=$(docker compose logs cuenta 2>&1 | grep -c "leida de Postgres")
  echo "Lecturas a Postgres: $((despues-antes))"
done
SIN CACHE - corrida 1
Consultas=100 promedio=26.56 ms mediana=23.13 ms p95=42.17 ms errores=0
Lecturas a Postgres: 100
SIN CACHE - corrida 2
Consultas=100 promedio=24.32 ms mediana=22.52 ms p95=37.33 ms errores=0
Lecturas a Postgres: 100
SIN CACHE - corrida 3
Consultas=100 promedio=23.59 ms mediana=21.73 ms p95=38.89 ms errores=0
Lecturas a Postgres: 100
```

Figura 13: Tres corridas sin caché: cien lecturas a PostgreSQL en cada una.

2.7 Bitácora de avisos en MongoDB

Se completó `BitacoraDeAvisos` para construir un documento por destinatario. Los campos comunes son el identificador, evento, transferencia, CLABE, tipo, monto, saldo, fecha y canal. El monto y el saldo se guardaron como `Decimal128`; la fecha corresponde a la del evento.

El identificador se formó como `eventoId:clabe`. Un evento genera hasta dos avisos, por lo que el identificador del evento por sí solo no sería suficiente. Cada documento se inserta por separado, y una `DuplicateKeyException` se registra como aviso existente, sin detener la inserción del otro destinatario.

El consumidor de RabbitMQ se conectó a `bitacora.registrar(evento)`. Los nuevos avisos de transferencias se escriben en MongoDB; no se añadió una escritura simultánea en PostgreSQL.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ docker compose logs --since 5m notificacion | grep "indice"
notificacion-1  | 2026-11-14T02:34:41.651Z  INFO 1 --- [notificacion] [           main] m.m.n.avisos.BitacoraDeAvisos
 : [avisos] indice clabe_fecha listo en avisos
```

Figura 14: Creación del índice clabe_fecha al iniciar notificación.

Se transfirieron $10 de Ana a Beto y $5 de Ana a Carla. Hubo cuatro documentos: dos SMS de Ana, un correo de Beto y un push de Carla. Los dos SMS son esperados, porque Ana participó como emisora en las dos transferencias.

```
{
  _id: '8b71dfd7-213e-46e8-8705-f7951cf679d3:002180000000000001',
  eventoId: '8b71dfd7-213e-46e8-8705-f7951cf679d3',
  transferenciaId: Long('35'),
  clabe: '002180000000000001',
  tipo: 'TRANSFERENCIA_ENVIADA',
  monto: Decimal128('10'),
  saldo: Decimal128('770.00'),
  fecha: ISODate('2026-11-14T02:37:55.276Z'),
  canal: 'SMS',
  telefono: '+52 55 5555 0101',
  texto: 'TRANSFERENCIA_ENVIADA por 10, saldo resultante 770.00'
}
```

Figura 15: Documento SMS de Ana: teléfono y texto.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --quiet --eval \
'db.avisos.find({clabe: {$in: ["002180000000000001", "002180000000000002", "002180000000000003"]}}).sort({fecha: 1}).forEach(d =
> printjson(d))'
{
  _id: '8b71dfd7-213e-46e8-8705-f7951cf679d3:002180000000000002',
  eventoId: '8b71dfd7-213e-46e8-8705-f7951cf679d3',
  transferenciaId: Long('35'),
  clabe: '002180000000000002',
  tipo: 'TRANSFERENCIA_RECIBIDA',
  monto: Decimal128('10'),
  saldo: Decimal128('730.00'),
  fecha: ISODate('2026-11-14T02:37:55.276Z'),
  canal: 'CORREO',
  correo: 'beto@correo.mx',
  asunto: 'TRANSFERENCIA_RECIBIDA',
  cuerpo: 'TRANSFERENCIA_RECIBIDA por 10, saldo resultante 730.00'
}
```

Figura 16: Documento de correo de Beto: dirección, asunto y cuerpo.

```
{
  _id: '13135412-2499-4817-a726-539e182abc5b:002180000000000003',
  eventoId: '13135412-2499-4817-a726-539e182abc5b',
  transferenciaId: Long('36'),
  clabe: '002180000000000003',
  tipo: 'TRANSFERENCIA_RECIBIDA',
  monto: Decimal128('5'),
  saldo: Decimal128('305.00'),
  fecha: ISODate('2026-11-14T02:37:57.027Z'),
  canal: 'PUSH',
  token: 'fcm:demo-002180000000000003',
  titulo: 'TRANSFERENCIA_RECIBIDA',
  cuerpo: 'TRANSFERENCIA_RECIBIDA por 5, saldo resultante 305.00'
}
```

Figura 17: Documento push de Carla: token, título y cuerpo.

Las tres formas conservan los mismos campos comunes, pero agregan solo los datos que necesita cada canal. Esto permite leer un aviso completo sin tener columnas vacías para los otros canales.

2.8 Consulta de los últimos diez avisos

Se programó la consulta por CLABE, con fecha descendente y límite de diez. Para comprobar el límite se hicieron doce transferencias de $1 de Ana a Beto. Después se solicitó `GET /api/notificaciones/002180000000000002`.

La respuesta del endpoint se guardó y se mostró como un resumen de canal, fecha e identificador de transferencia. Devolvió exactamente diez avisos de correo, de la transferencia 48 a la 39, del más nuevo al más viejo.

```
MINGW64:/c/Users/cecis/Desktop/mexi-banco-s11
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/notificaciones/002180000000000002 \
  > avisos-beto.json

docker compose exec -T mongo mongosh --nodb --quiet --eval '
const fs = require("fs");
const avisos = JSON.parse(fs.readFileSync("/dev/stdin", "utf8"));
print("Avisos devueltos por el endpoint: " + avisos.length);
avisos.forEach((a, i) => printjson({
  posicion: i + 1,
  canal: a.canal,
  fecha: a.fecha,
  transferenciaId: a.transferenciaId
}));
' < avisos-beto.json
Avisos devueltos por el endpoint: 10
{
  posicion: 1,
  canal: 'CORREO',
  fecha: '2026-11-14T02:44:48.131+00:00',
  transferenciaId: 48
}
{
  posicion: 2,
  canal: 'CORREO',
  fecha: '2026-11-14T02:44:47.784+00:00',
  transferenciaId: 47
}
{
  posicion: 3,
  canal: 'CORREO',
  fecha: '2026-11-14T02:44:47.450+00:00',
  transferenciaId: 46
}
{
  posicion: 4,
  canal: 'CORREO',
  fecha: '2026-11-14T02:44:47.101+00:00',
  transferenciaId: 45
}
```

Figura 18: Respuesta del endpoint: conteo de diez avisos y primeras posiciones.

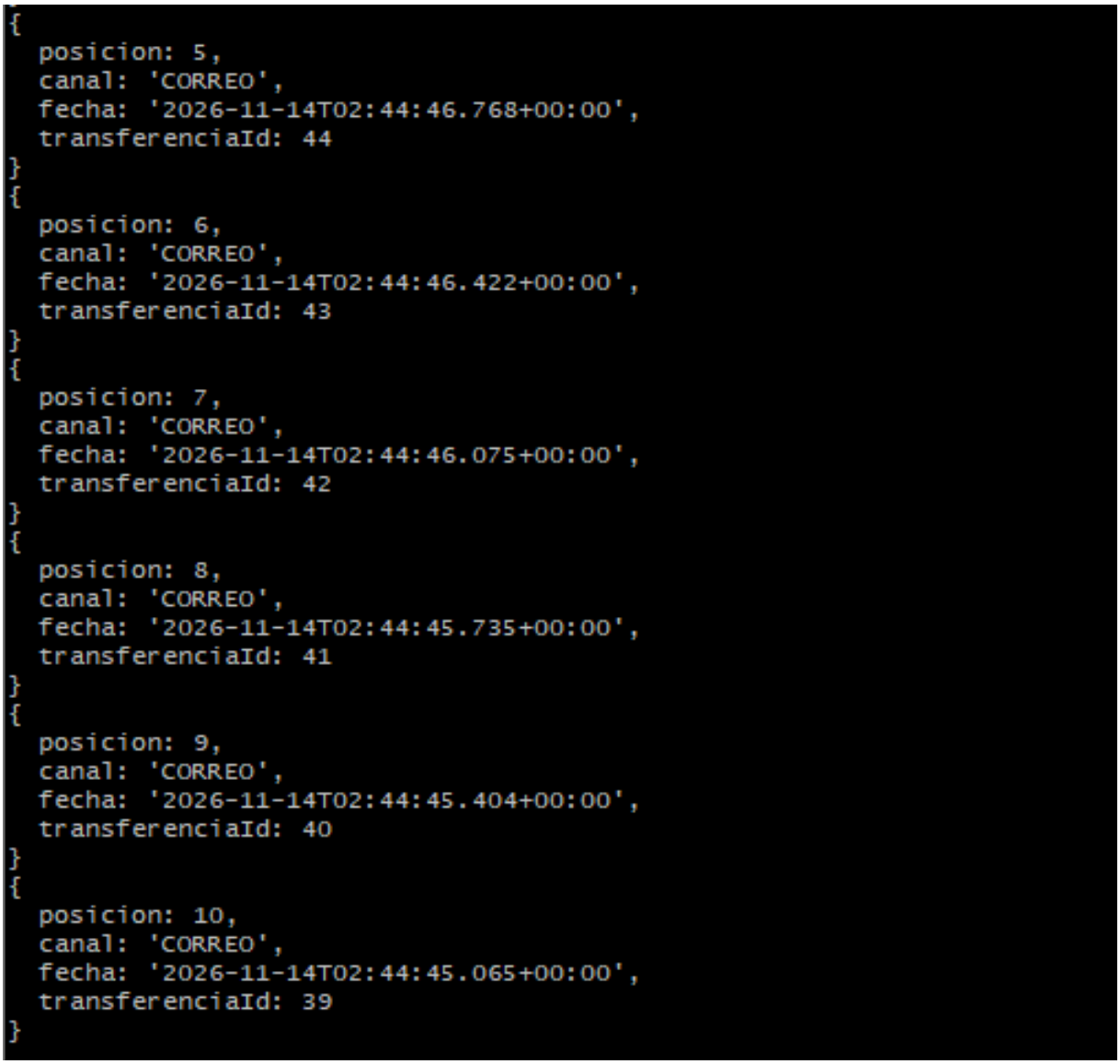


Figura 19: Continuación de la misma captura: posiciones finales de la respuesta.

2.9 Índice y planes de ejecución

Se creó el índice compuesto `clabe_fecha`, con CLABE ascendente y fecha descendente. Este orden corresponde a la búsqueda por igualdad de CLABE y al orden de lectura solicitado.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --quiet --eval \
'db.avisos.getIndexes()'
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { clabe: 1, fecha: -1 }, name: 'clabe_fecha' }
]
```

Figura 20: Índices de la colección: _id_ y clabe_fecha.

Se comparó el mismo filtro, orden y límite con `explain(.executionStats")`. En la segunda consulta se añadió `hint({$natural:1})` para obligar un recorrido de la colección.

Consulta	Devueltos	Claves examinadas	Documentos examinados
Con índice	10	10	10
Sin índice	10	0	28

Con índice aparecieron las etapas `IXSCAN`, `FETCH` y `LIMIT`; sin él aparecieron `COLLSCAN` y `SORT`. El índice permitió leer los diez documentos necesarios sin recorrer los 28 ni añadir una etapa de ordenamiento.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --quiet --eval '
const e = db.avisos.find({clabe: "002180000000000002"})
  .sort({fecha: -1}).limit(10).hint({$natural: 1})
  .explain("executionStats");
print("CONSULTA SIN INDICE");
printjson(e.queryPlanner.winningPlan);
printjson({
  nReturned: e.executionStats.nReturned,
  totalKeysExamined: e.executionStats.totalKeysExamined,
  totalDocsExamined: e.executionStats.totalDocsExamined
});
'
CONSULTA SIN INDICE
{
  isCached: false,
  stage: 'SORT',
  sortPattern: {
    fecha: -1
  },
  memLimit: 104857600,
  limitAmount: 10,
  type: 'simple',
  inputStage: {
    stage: 'COLLSCAN',
    filter: {
      clabe: {
        '$eq': '002180000000000002'
      }
    },
    direction: 'forward'
  }
}
{
  nReturned: 10,
  totalKeysExamined: 0,
  totalDocsExamined: 28
}
```

Figura 21: Sin índice, MongoDB examinó los 28 documentos y ordenó los resultados.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --quiet --eval '
const e = db.avisos.find({clabe: "002180000000000002"})
  .sort({fecha: -1}).limit(10).explain("executionStats");
print("CONSULTA CON INDICE");
printjson(e.queryPlanner.winningPlan);
printjson({
  nReturned: e.executionStats.nReturned,
  totalKeysExamined: e.executionStats.totalKeysExamined,
  totalDocsExamined: e.executionStats.totalDocsExamined
});
'
CONSULTA CON INDICE
{
  isCached: false,
  stage: 'LIMIT',
  limitAmount: 10,
  inputStage: {
    stage: 'FETCH',
    inputStage: {
      stage: 'IXSCAN',
      keyPattern: {
        clabe: 1,
        fecha: -1
      },
      indexName: 'clabe_fecha',
      isMultiKey: false,
      multiKeyPaths: {
        clabe: [],
        fecha: []
      },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: {
        clabe: [
          '["002180000000000002", "002180000000000002"]'
        ],
        fecha: [
          '[MaxKey, MinKey]'
        ]
      }
    }
  }
}
{
  nReturned: 10,
  totalKeysExamined: 10,
  totalDocsExamined: 10
}
```

Figura 22: Plan con clabe_fecha: diez claves y diez documentos examinados.

2.10 Evento duplicado e idempotencia

Se publicó dos veces el mismo evento desde la API de RabbitMQ, con el identificador siguiente:

`11111111-2222-4333-8444-555555555555`.

Se utilizaron las CLABEs de prueba terminadas en 091 y 092. Esta publicación no aplicó cargos ni abonos: únicamente probó el consumo del evento.

Las dos publicaciones respondieron `routed:true`. Después de procesarlas quedaron dos avisos, no cuatro. El segundo intento encontró los mismos identificadores y registró `ya estaba` para cada destinatario. Así se comprobó que una entrega repetida no duplica los documentos.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ for intento in 1 2; do
  echo "Publicacion $intento"

  curl -s -u mexibanco:mexibanco \
    -X POST http://localhost:15672/api/exchanges/%2F/transferencias/publish \
    -H 'Content-Type: application/json' \
    -d '{"properties":{"delivery_mode":2,"content_type":"application/json"},"routing_key":"transferencia.realizada","payload_enc
oding":"string","payload":"{\"eventoId\":\"11111111-2222-4333-8444-555555555555\",\"tipo\":\"TransferenciaRealizada\",\"transfer
enciaId\":9001,\"clabeOrigen\":\"002180000000000091\",\"clabeDestino\":\"002180000000000092\",\"monto\":25,\"saldoOrigen\":475,\
"saldoDestino\":525,\"fecha\":\"2026-10-31T13:00:00Z\"}"}'
  echo
done
Publicacion 1
{"routed":true}
Publicacion 2
{"routed":true}
```

Figura 23: El mismo evento fue enviado dos veces y ambas publicaciones se enrutaron.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ echo "Avisos guardados para el evento duplicado:"

docker compose exec -T mongo mongosh notificacion --quiet --eval \
'db.avisos.countDocuments({eventoId: "11111111-2222-4333-8444-555555555555"})'

docker compose logs --since 5m notificacion | grep "11111111" | grep "ya estaba"
Avisos guardados para el evento duplicado:
2
notificacion-1  | 2026-11-14T02:54:41.946Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos
 : [avisos] 11111111-2222-4333-8444-555555555555:002180000000000091 ya estaba: evento repetido, no se duplica
notificacion-1  | 2026-11-14T02:54:41.948Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos
 : [avisos] 11111111-2222-4333-8444-555555555555:002180000000000092 ya estaba: evento repetido, no se duplica
```

Figura 24: Conteo de dos avisos y mensajes de duplicado para los dos destinatarios.

2.11 Dato viejo: la caché muestra, pero no autoriza

El saldo de Ana era de $753. Se consultó para llenar Redis y luego se cambió directamente en PostgreSQL a $5, sin pasar por el servicio ni por la invalidación. Se observaron el TTL y la respuesta del GET aproximadamente cada cinco segundos.

Segundos desde el cambio	TTL observado	Saldo mostrado
1	22	$753
8	16	$753
14	10	$753
19	4	$753
25	-2	$5

El último saldo viejo se observó a los 19 segundos y el primero actualizado a los 25. Por la frecuencia de muestreo, la transición se ubica entre esas dos observaciones; no se puede afirmar un segundo exacto. El TTL restante era menor a 30 desde el inicio del ciclo, porque ya había transcurrido tiempo entre llenar la caché y modificar PostgreSQL.

Mientras la copia conservó $753, su `leidaEn` fue el mismo. Al caducar, cambió de aproximadamente 02:58:30 a 02:59:02 UTC, porque se volvió a leer PostgreSQL. El intento de transferir $100 fue rechazado con HTTP 422 y el mensaje de saldo insuficiente.

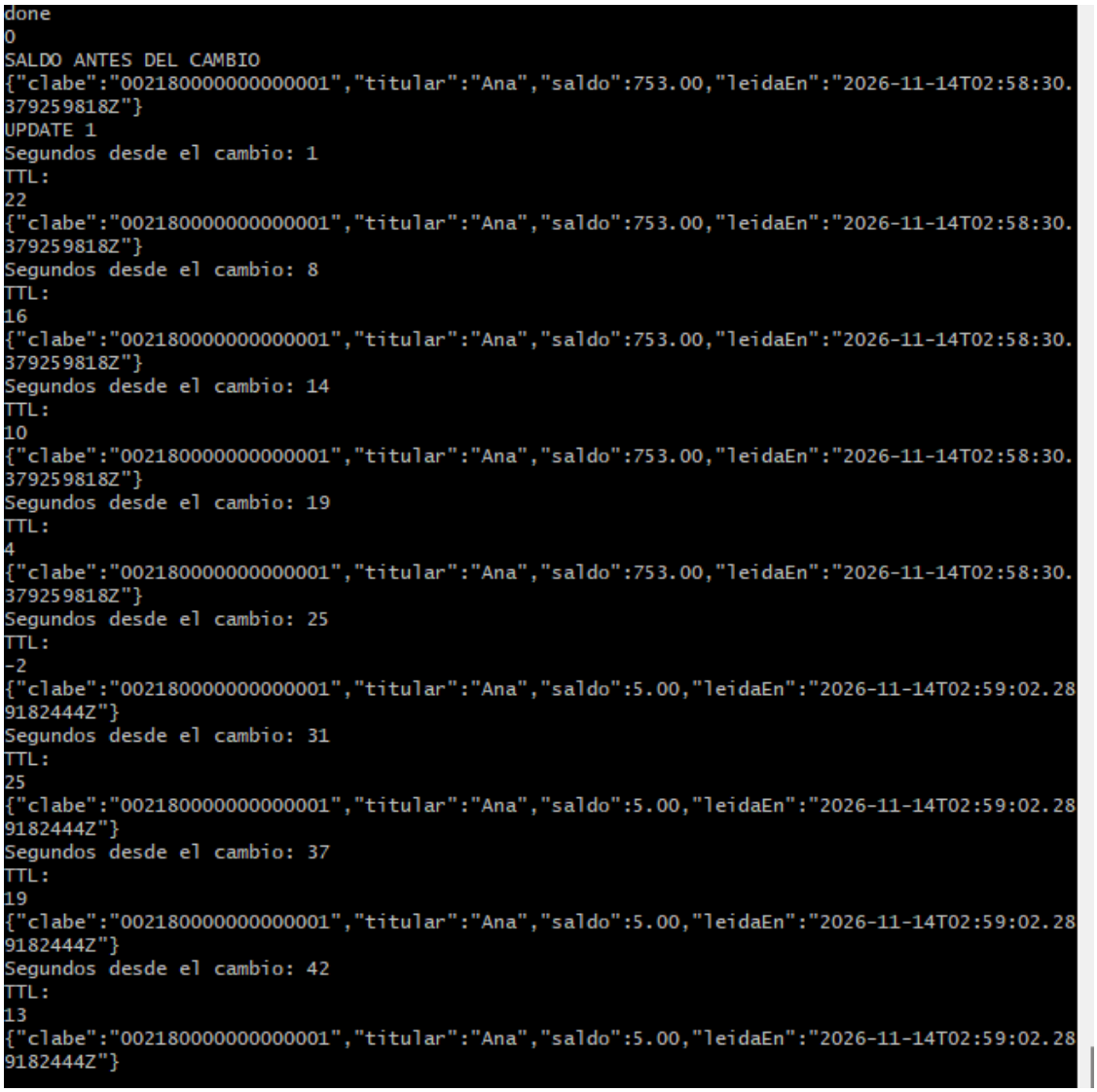


Figura 25: Detalle de la captura original: el saldo viejo desaparece entre los segundos 19 y 25.

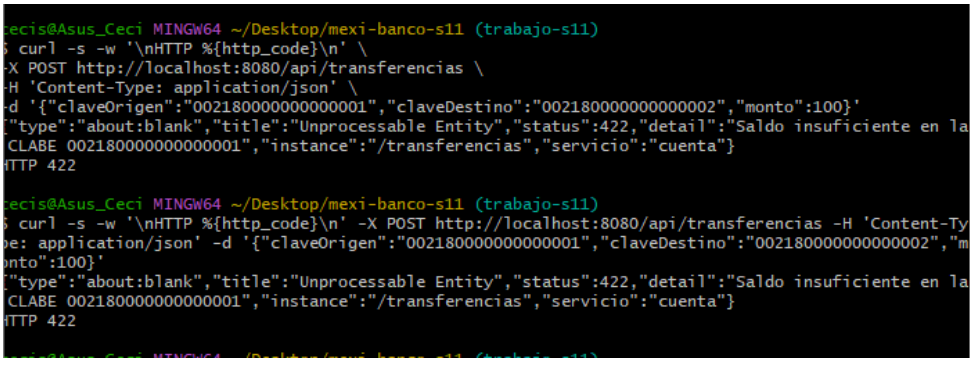


Figura 26: Detalle de la misma captura: transferencia rechazada con HTTP 422.

Al terminar se restauraron los $753 en PostgreSQL y se borró la copia de Redis. La siguiente consulta confirmó el saldo restaurado con HTTP 200. Este cambio directo fue únicamente una prueba controlada; no se utilizó como mecanismo normal para mover dinero.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T db psql -U mexibanco -d cuenta -v ON_ERROR_STOP=1 -c \
"UPDATE cuenta SET saldo = 753.00 WHERE clabe = '002180000000000001';"

docker compose exec -T redis redis-cli DEL cuentas::002180000000000001

curl -s -w '\nHTTP %{http_code}\n' \
http://localhost:8080/api/cuentas/002180000000000001
UPDATE 1
0
{"clabe":"002180000000000001","titular":"Ana","saldo":753.00,"leidaEn":"2026-11-14T03:00:44.
701193022Z"}
HTTP 200
```

Figura 27: Restauración del saldo original después de la prueba de dato viejo.

2.12 Reto: funcionamiento con Redis apagado

Primero se apagó Redis con la implementación básica. El GET devolvió HTTP 500, aun cuando PostgreSQL seguía disponible. Esto mostró que una copia temporal estaba afectando la disponibilidad de la consulta.

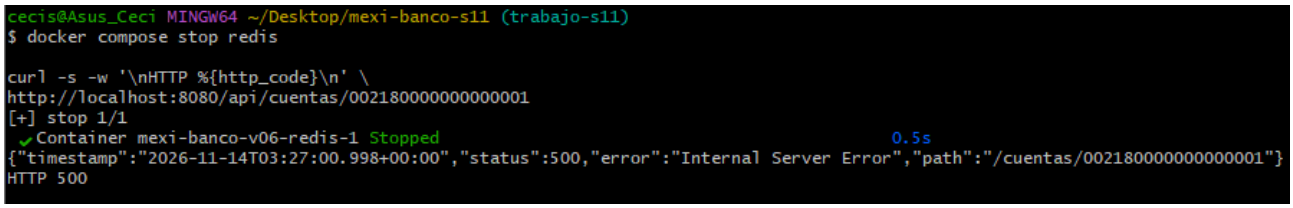


Figura 28: Comportamiento inicial: sin Redis, el GET respondía HTTP 500.

Se implementó una variante de `RedisCache` que atrapa los errores de las operaciones de caché y registra el nombre de la operación, la clave y la excepción. Si falla GET, devuelve un miss y la lectura sigue en PostgreSQL. Si falla SET o DEL, registra el error y deja continuar la operación.

El administrador sigue siendo transaccional. El manejo de errores se encuentra en la caché interna, por debajo del decorador transaccional, de modo que también cubre el DEL diferido hasta después del commit. Un manejador situado solamente alrededor de la anotación no necesariamente cubriría ese callback posterior. Esto evita devolver un error por caché cuando el cargo ya fue confirmado.

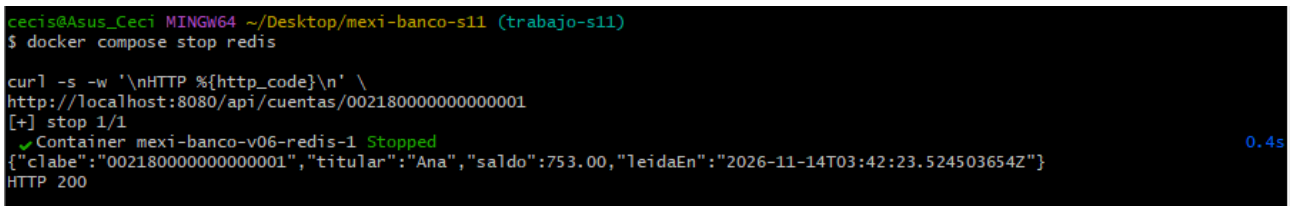


Figura 29: Con el cambio, el GET devolvió $753 y HTTP 200 sin Redis.

Con Redis todavía apagado se transfirió $1 de Ana a Beto. La transferencia 51 respondió HTTP 201 y quedó en `COMPLETADA`. Ana pasó de $753 a $752 y Beto de $742 a $743. La suma de ambas cuentas se conservó en $1495.

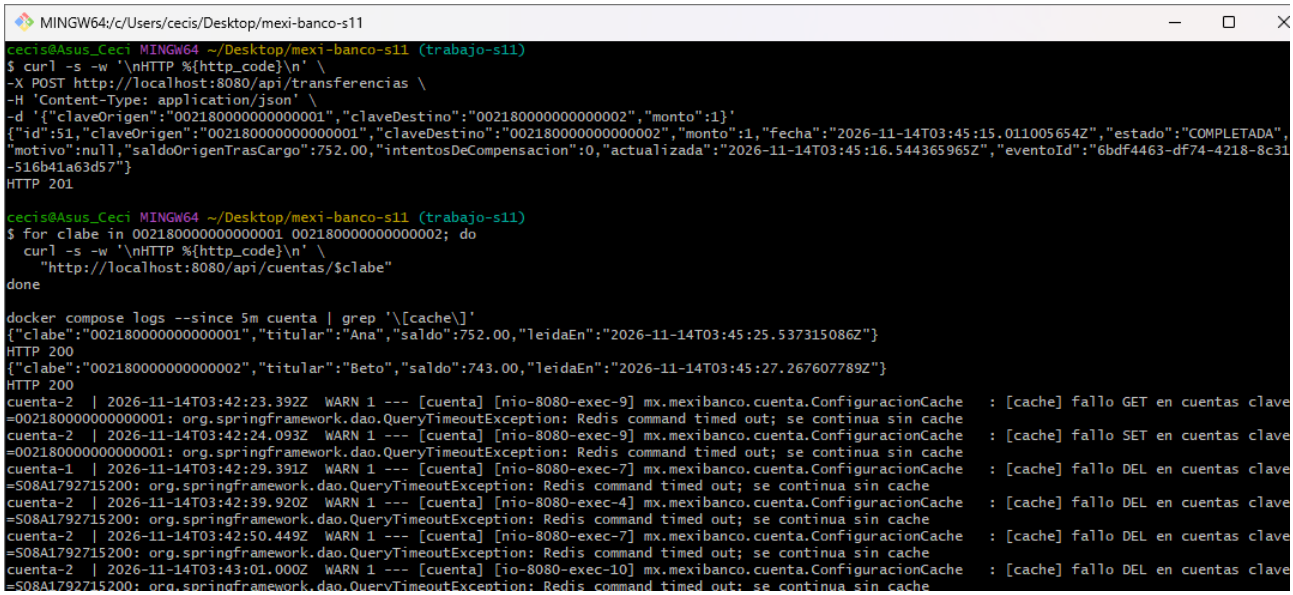


Figura 30: Transferencia completada y consultas HTTP 200 sin Redis.

Los logs mostraron fallos de GET, SET y DEL, pero la operación de dinero no se interrumpió. Las peticiones pueden tardar más por el timeout de Redis y la lectura a PostgreSQL. La tolerancia a fallos no elimina el costo de esperar al cliente de caché.

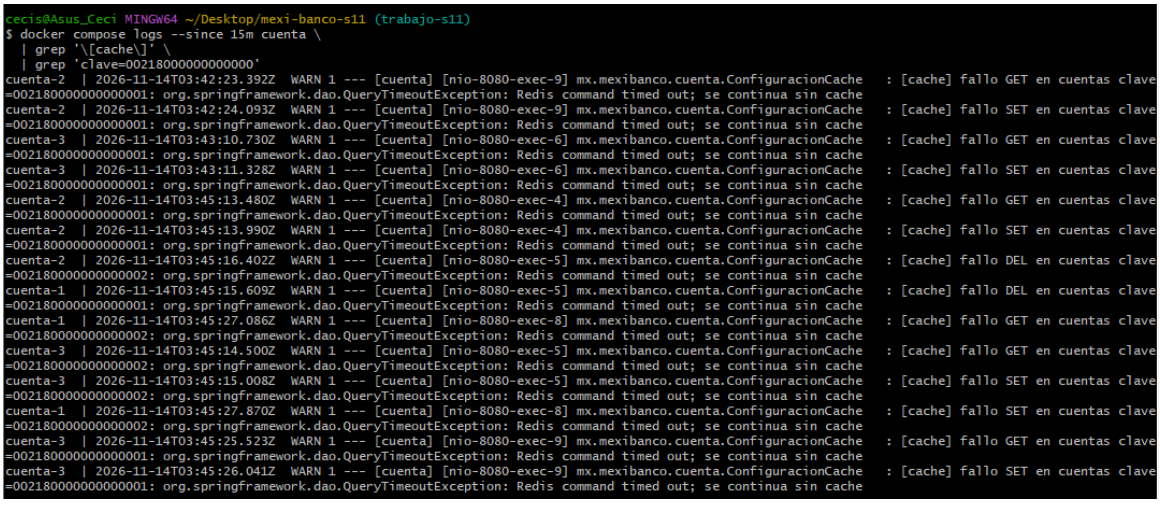


Figura 31: Registro de las operaciones de caché que fallaron durante la caída.

Al encender Redis, respondió PONG. Dos consultas consecutivas devolvieron $752, HTTP 200 y el mismo `leidaEn`. El GET directo de Redis mostró el JSON y el TTL restante fue de 27 segundos.

```
cecis@Asus_Ceci MINGW64 ~/Desktop/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli ping

for i in 1 2; do
  curl -s -w '\nHTTP %{http_code}\n' \
    http://localhost:8080/api/cuentas/002180000000000001
done

docker compose exec -T redis redis-cli GET cuentas::002180000000000001
docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
PONG
{"clabe":"002180000000000001","titular":"Ana","saldo":752.00,"leidaEn":"2026-11-14T03:57:56.806301264Z"}
HTTP 200
{"clabe":"002180000000000001","titular":"Ana","saldo":752.00,"leidaEn":"2026-11-14T03:57:56.806301264Z"}
HTTP 200
{"clabe":"002180000000000001","titular":"Ana","saldo":752.00,"leidaEn":"2026-11-14T03:57:56.806301264Z"}
27
```

Figura 32: Redis recuperado: la copia volvió a guardarse y a compartirse entre consultas.

La solución privilegia la disponibilidad cuando falla la caché. Si un DEL no llega a ejecutarse mientras todavía existe una copia, esta puede permanecer hasta vencer su TTL; por eso la autorización nunca depende de Redis. En la prueba se detuvo Redis sin persistencia y la copia se reconstruyó al regresar.

3. Preguntas de cierre

1. ¿Por qué MongoDB para los avisos y no para el saldo?

Los avisos tienen campos distintos según el canal. El SMS lleva teléfono y texto, el correo tiene dirección y asunto, y el push utiliza un token. MongoDB permitió guardar cada forma como un documento completo y leer los últimos avisos por CLABE. El saldo, en cambio, necesita transacciones y control de concurrencia para que los cargos y abonos no se contradigan. Por eso se mantuvo en PostgreSQL.

2. ¿Qué hit ratio se obtuvo y qué aportó la caché?

En A obtuve 99 % y en B 83.33 %. En B bajó porque las transferencias borraron las copias y también agregaron consultas de origen y destino. Los promedios fueron de 18.34 a 20.63 ms con caché y de 23.59 a 26.56 ms sin ella. Sí hubo una mejora de velocidad, pero lo más claro fue reducir las lecturas a PostgreSQL de cien a una por corrida. Se ganó margen de capacidad en la base, aunque no se midió su límite con usuarios concurrentes.

3. ¿Qué pasaría si el cargo se autorizara con la caché?

Podría aprobarse una transferencia con un saldo que ya no existe. En la prueba, la consulta todavía mostraba $753 aunque PostgreSQL ya tenía $5. El intento de transferir $100 respondió HTTP 422 porque el cargo leyó el saldo real. Si hubiera confiado en la copia, podría haber autorizado un monto mayor al disponible.

4. ¿Por qué DEL y por qué después del commit?

Se borra la copia para que la siguiente consulta reconstruya el saldo desde PostgreSQL. Un SET con el saldo de un movimiento podría sobrescribir un valor más reciente si hay operaciones concurrentes. A su vez, el borrado debe ocurrir después del commit: si se hace antes, otra consulta podría leer el saldo anterior y volver a guardarlo en Redis antes de que se confirme el cambio.

4. Conclusiones

En este laboratorio observé que usar varias bases tiene sentido cuando cada una atiende una necesidad concreta. PostgreSQL conservó el dinero, MongoDB permitió representar los avisos según su canal y Redis redujo las lecturas repetidas del saldo.

Las pruebas también dejaron ver que una copia no siempre refleja el estado actual. El TTL permitió que el saldo viejo caducara, pero lo que evitó un cargo incorrecto fue mantener la autorización en PostgreSQL. A su vez, el identificador por evento y CLABE permitió recibir el mismo mensaje sin duplicar avisos.

Con el reto de Redis apagado comprobé que la caché no debe impedir las operaciones del banco. Fue necesario manejar también los errores del borrado posterior al commit, porque un fallo de caché no significa que el movimiento de dinero se haya deshecho. La mejora de rendimiento fue útil, siempre que se mantuviera clara la diferencia entre la verdad y su copia.

Material de referencia

Guía del Laboratorio S11, *Mexi Banco: una verdad, varias copias*, Sistemas Distribuidos, Facultad de Ingeniería, UNAM, semestre 2027-1. Código base: https://github.com/OscarRuiz21/mexi-banco, rama `06-poliglota-base`.